

Санкт-Петербургский Национальный Исследовательский
Университет Информационных Технологий, Механики и
Оптики

Факультет Инфокоммуникационных Технологий

Практическая работа №6

«Работа с сокетами»

Выполнила:

Соколова Дарья Максимовна

Группа: К3323

Проверила:

Марченко Елена Вадимовна

Санкт-Петербург,

2025

Цель работы

Получить практические навыки и умения реализации web-сервисов и использования сокетов.

Задачи

1. Реализовать клиентскую и серверную часть приложения. Клиент отправляет серверу сообщение «Hello, server». Сообщение должно отразиться на стороне сервера. Сервер в ответ отправляет клиенту сообщение «Hello, client». Сообщение должно отобразиться у клиента;
2. Реализовать клиентскую и серверную часть приложения. Клиент запрашивает у сервера выполнение математической операции, параметры, которые вводятся с клавиатуры. Сервер обрабатывает полученные данные и возвращает результат клиенту;
3. Реализовать серверную часть приложения. Клиент подключается к серверу. В ответ клиент получает http-сообщение, содержащее html-страницу, которую сервер подгружает из файла index.html;
4. Реализовать многопользовательский чат.

Ход работы

Задание 1

Для выполнения первого задания создана серверная и клиентская часть приложения. Для работы с сетевыми соединениями используется библиотека **socket**.

Серверная часть представлена на рисунке 1. Здесь сначала создается сокет сервера, привязывается к адресу и порту, ожидается подключение одного клиента. Затем сокет принимает входящее подключение и выводит информацию о подключенном клиенте. Сервер получает сообщение от клиента, которые выводятся в консоли, а затем формируется и отправляется ответ для клиента, соединение с клиентом и сокета сервера закрывается.

На рисунке 2 представлена клиентская часть. Здесь сначала создается сокет клиента, затем он подключается к серверу по заданному адресу и порту. Формируется и отправляется сообщение на сервер. Затем клиент получает ответ от сервера и оно выводится в консоль. Сокет клиента закрывается.

Результат работы представлен на рисунках 3-4.

```
import socket

def start_server():
    server_socket = socket.socket()

    server_socket.bind(('localhost', 8084))
    server_socket.listen(1)
    print("Сервер запущен и ожидает подключения...")

    conn, address = server_socket.accept()
    print(f"Подключен к {address}")

    data = conn.recv(1024).decode('utf-8')
    print(f"Сообщение от клиента: {data}")

    response = "Hello, client"
    conn.send(response.encode('utf-8'))

    conn.close()
    server_socket.close()

if __name__ == "__main__":
    start_server()
```

Рисунок 1 – Серверная часть

```
import socket

def start_client():
    client_socket = socket.socket()

    client_socket.connect(('localhost', 8084))

    message = "Hello, server"
    client_socket.send(message.encode('utf-8'))

    response = client_socket.recv(1024).decode('utf-8')
    print(f"Ответ от сервера: {response}")

    client_socket.close()

if __name__ == "__main__":
    start_client()
```

Рисунок 2 – Клиентская часть

```
• (base) darya@Darias-MacBook-Air task1 % python server.py
Сервер запущен и ожидает подключения...
Подключен к ('127.0.0.1', 57460)
Сообщение от клиента: Hello, server
```

Рисунок 3 – Получение сообщения от клиента

```
• (base) darya@Darias-MacBook-Air task1 % python client.py
Ответ от сервера: Hello, client
```

Рисунок 4 – Получение сообщения от сервера

Задание 2

Во 2 задании клиент отправляет данные о параллелограмме на сервер, а сервер вычисляет площадь параллелограмма и отправляет результат обратно клиенту.

Клиентская часть представлена на рисунке 5. Сначала создается новый сокет для клиента и устанавливается соединение с сервером. Затем пользователь вводит основание и высоту параллелограмма и эти данные отправляются на сервер. Клиент получает ответ и выводит его на экран, соединение закрывается.

Серверная часть представлена на рисунке 6. Здесь отдельно вынесена функция для вычисления площади параллелограмма. В основной функции создается сокет, который связывается с адресом и портом и ожидает подключения клиента. Сервер принимает подключение, получает, декодирует и разделяет данные, а затем вычисляет площадь и отправляет результат. Соединение с клиентом и сокет сервера закрываются.

Результат работы представлен на рисунках 7-8.

```

import socket

def start_client():
    client_socket = socket.socket()
    client_socket.connect(('localhost', 1200))

    base = input("Введите основание параллелограмма: ")
    height = input("Введите высоту параллелограмма: ")

    client_socket.send(f"{base},{height}".encode('utf-8'))

    response = client_socket.recv(1024).decode('utf-8')
    print('Результат: ', response)

    client_socket.close()

if __name__ == "__main__":
    start_client()

```

Рисунок 5 – Клиентская часть

```

import socket

def calculate_area(base, height):
    return base * height

def start_server():
    server_socket = socket.socket()
    server_socket.bind(('localhost', 1200))
    server_socket.listen(1)

    print("Сервер запущен и ожидает подключения...")

    conn, address = server_socket.accept()
    print(f"Подключен к {address}")

    data = conn.recv(1024).decode('utf-8')
    base, height = map(float, data.split(','))

    area = calculate_area(base, height)
    conn.send(f"Площадь параллелограмма: {area}".encode('utf-8'))

    conn.close()
    server_socket.close()

if __name__ == "__main__":
    start_server()

```

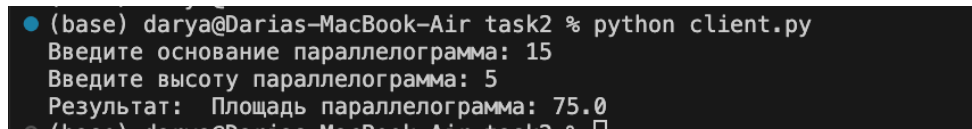
Рисунок 6 – Серверная часть

```

(base) darya@Darias-MacBook-Air task2 % python server.py
Сервер запущен и ожидает подключения...
Подключен к ('127.0.0.1', 57792)

```

Рисунок 7 – Ожидание подключения клиента на сервере



```
(base) darya@Darias-MacBook-Air task2 % python client.py
Введите основание параллелограмма: 15
Введите высоту параллелограмма: 5
Результат: Площадь параллелограмма: 75.0
```

Рисунок 8 – Получение сообщения от сервера

Задание 3

В 3 задании реализована серверная часть приложения, при котором клиент отправляет запрос, а сервер пытается открыть его содержимое и отправить обратно в качестве HTTP-ответа.

Серверная часть представлена на рисунке 9. Здесь создается новый сокет для сервера, он привязывается к адресу и порту, и устанавливается в режим ожидания подключения клиента. Когда клиент (например, веб-браузер) отправляет запрос, сервер пытается открыть файл `index.html`, прочитать его содержимое и отправить его обратно в качестве HTTP-ответа. Если файл не удастся открыть (например, если его нет), сервер отправляет сообщение об ошибке 500.

Пример выполнения представлен на рисунке 10, отображение html-файла представлено на рисунке 11.

```
import socket

def start_server():
    server_socket = socket.socket()
    server_socket.bind(('localhost', 3005))
    server_socket.listen(1)

    print("Сервер запущен и ожидает подключения...")

    while True:
        conn, address = server_socket.accept()
        print(f"Подключен к {address}")

        request = conn.recv(1024).decode('utf-8')
        print(f"Запрос: {request}")

        try:
            with open('index.html', 'r', encoding='utf-8') as file:
                html_content = file.read()

            response = "HTTP/1.1 200 OK\r\n"
            response += "Content-Type: text/html; charset=utf-8\r\n"
            response += f"\r\n{html_content}"
        except Exception as e:
            response = "HTTP/1.1 500 Internal Server Error\r\n"
            response += "Content-Type: text/html; charset=utf-8\r\n"
            response += "\r\n<h1>500 Internal Server Error</h1>"
            print(f"Ошибка при чтении файла: {e}")

        conn.send(response.encode('utf-8'))
        conn.close()

if __name__ == "__main__":
    start_server()
```

Рисунок 9 – Серверная часть

```
(base) darya@Darias-MacBook-Air task3 % python server.py
Сервер запущен и ожидает подключения...
Подключен к ('127.0.0.1', 58824)
Запрос: GET / HTTP/1.1
Host: localhost:3005
```

Рисунок 10 – Пример выполнения

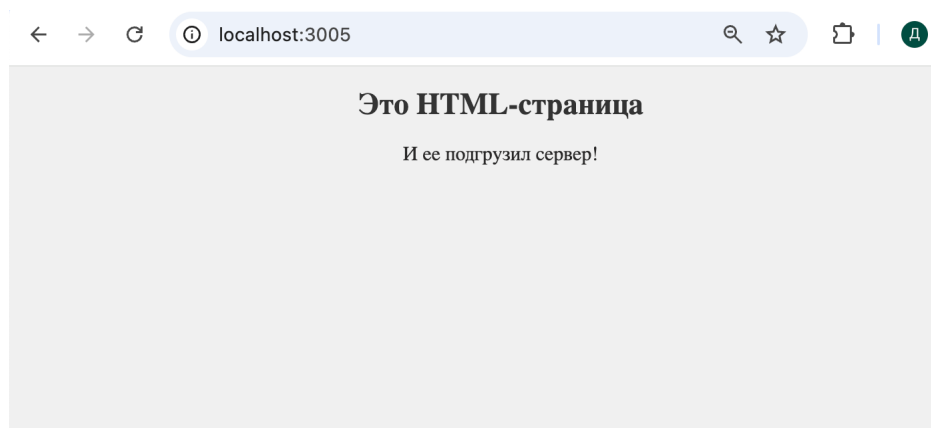


Рисунок 11 – Отображение html-страницы

Задание 4

В 4 задании необходимо создать многопользовательский чат.

На рисунке 12 представлена серверная часть. Здесь используется модуль **threading** для создания и управления потоками, что позволяет обрабатывать сразу несколько клиентов одновременно. В словаре `clients` ключами являются сокеты клиентов, а знаменами — их имена, позволяет отслеживать подключенных клиентов. Функция `handle_client` обрабатывает взаимодействие с конкретным клиентом, в основном цикле идет получение сообщения от клиента, которое передается остальным клиентам через `broadcast_message`. Удаление клиента реализовано в функции `remove_client`, где клиент удаляется из словаря `clients` и все остальные клиенты уведомляются о его отключении. Когда клиент подключается создается новый поток для обработки этого клиента.

На рисунке 13 представлена клиентская часть. Функция `receive_message` получает и отображает сообщения от сервера, функция `start_client` запускает клиента, здесь пользователь вводит свое имя, которое отправляется на сервер и создается новый поток для получения сообщений от сервера. В основном цикле клиент ожидает ввода сообщения и отправляет его на сервер. Если вводится `exit`, соединение закрывается.

На рисунках 14-17 представлен пример выполнения программы.


```

import socket
import threading

clients = {}

def handle_client(client_socket, client_address):
    print(f"[{client_address}] Подключен.")

    username = client_socket.recv(1024).decode('utf-8')
    clients[client_socket] = username
    broadcast_message(f"{username} присоединился к чату.", client_socket)

    while True:
        try:
            message = client_socket.recv(1024).decode('utf-8')
            if message:
                broadcast_message(f"{username}: {message}", client_socket)
            else:
                remove_client(client_socket)
                break
        except:
            remove_client(client_socket)
            break

def broadcast_message(message, sender_socket):
    for client in clients:
        if client != sender_socket:
            try:
                client.send(message.encode('utf-8'))
            except:
                remove_client(client)

def remove_client(client_socket):
    if client_socket in clients:
        username = clients[client_socket]
        del clients[client_socket]
        print(f"[{client_socket.getpeername()}] Отключен.")
        broadcast_message(f"{username} покинул чат.", client_socket)

def start_server():
    server_socket = socket.socket()
    server_socket.bind(('localhost', 1335))
    server_socket.listen(5)
    print("Сервер запущен и ожидает подключения...")

    while True:
        client_socket, client_address = server_socket.accept()
        threading.Thread(target=handle_client, args=(client_socket, client_address)).start()

if __name__ == "__main__":
    start_server()

```

Рисунок 12 – Серверная часть

```

import socket
import threading

def receive_messages(client_socket):
    while True:
        try:
            message = client_socket.recv(1024).decode('utf-8')
            if message:
                print(message)
            else:
                break
        except:
            print("Ошибка подключения к серверу.")
            break

def start_client():
    client_socket = socket.socket()
    client_socket.connect(('localhost', 1335))

    username = input("Введите ваше имя: ")
    client_socket.send(username.encode('utf-8'))

    threading.Thread(target=receive_messages, args=(client_socket,)).start()

    while True:
        message = input()
        if message.lower() == 'exit':
            break
        client_socket.send(message.encode('utf-8'))

    client_socket.close()

if __name__ == "__main__":
    start_client()

```

Рисунок 13 – Клиентская часть

```

(base) darya@Darias-MacBook-Air task4 % python server.py
Сервер запущен и ожидает подключения...
[('127.0.0.1', 62327)] Подключен.
[('127.0.0.1', 62377)] Подключен.
[('127.0.0.1', 62403)] Подключен.
[('127.0.0.1', 62403)] Отключен.

```

Рисунок 14 – Подключение клиентов к серверу

```

(base) darya@Darias-MacBook-Air task4 % python client.py
Введите ваше имя: Клиент 1
Клиент 2 присоединился к чату.
Клиент 3 присоединился к чату.
Привет всем!
Клиент 2: Привет-привет! Как дела?
Клиент 3: У меня хорошо!
Клиент 3: Всем пока!
Клиент 3 покинул чат.

```

Рисунок 15 – Пример общения клиентов - 1

```
○ (base) darya@Darias-MacBook-Air task4 % python client.py
Введите ваше имя: Клиент 2
Клиент 3 присоединился к чату.
Клиент 1: Привет всем!
Привет-привет! Как дела?
Клиент 3: У меня хорошо!
Клиент 3: Всем пока!
Клиент 3 покинул чат.
█
```

Рисунок 16 – Пример общения клиентов - 2

```
● (base) darya@Darias-MacBook-Air task4 % python client.py
Введите ваше имя: Клиент 3
Клиент 1: Привет всем!
Клиент 2: Привет-привет! Как дела?
У меня хорошо!
Всем пока!
exit
Ошибка подключения к серверу.
```

Рисунок 17 – Пример общения клиентов - 3

Вывод

В данной практической работе были получены практические навыки по реализации web-сервисов и использованию сокетов, было создано несколько клиентских и серверных частей приложения, а также многопользовательский чат.